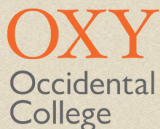


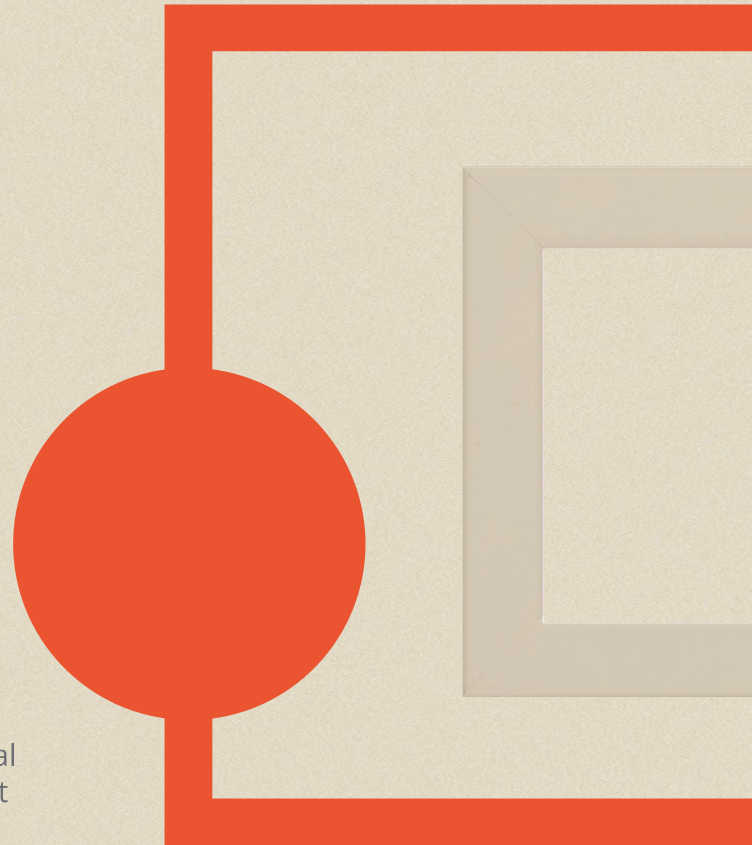
WebAssembly Benchmarking

Carly Venenciano
Occidental College
November 3, 2023



S3D

Software and Societal
Systems Department



01

Overview

02

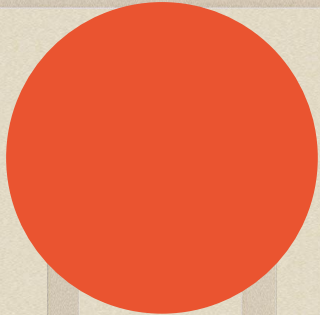
Methodology

03

**Results &
Extensions**

04

**Research
Experience**





01

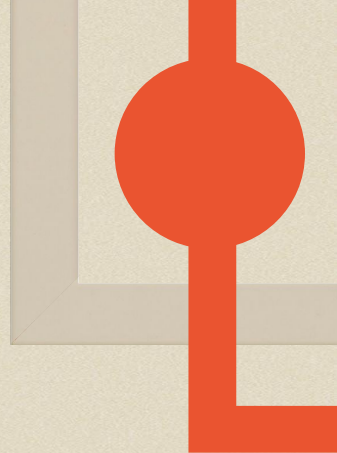
OVERVIEW

Researching Wasm engine
performance

WHAT IS WASM?



WebAssembly (Wasm) is a portable target for compiling various programming languages (Rust, C, C++) into bytecode to be executed on various operating systems.



Definitions



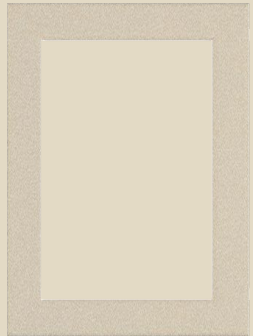
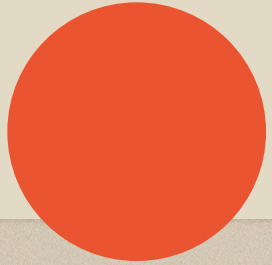
Engine

Also known as a virtual machine (VM), an engine executes program in one language on a host machine



Benchmark

A benchmark is a program and input that represents a workload of interest



Overview

- Builds off research done on the Wizard Engine
- Compare engine performance across three timed metrics
 - Total Time: engine startup to program completion
 - Main Time: execution of “main” function
 - Set Up Time: startup-related program up to first bytecode executed
- Many diverse techniques to executing Wasm code
 - Multidimensional tradeoff space; ex. Less time to compile vs. better quality execution code
 - Certain trade-offs more valuable depending on project
- Benchmarking is a tool for Wasm users to see the tradeoffs for their specific project

Engine

Interpreter

a tier of an engine that decodes and executes each byte of bytecode individually

Compiler

a tier of an engine that translates input bytecode to native code and executes the native code

Tiers

execution strategies

Benchmark

Suite

a collection of related programs in a benchmark

Line Item

an individual program within a benchmark suite

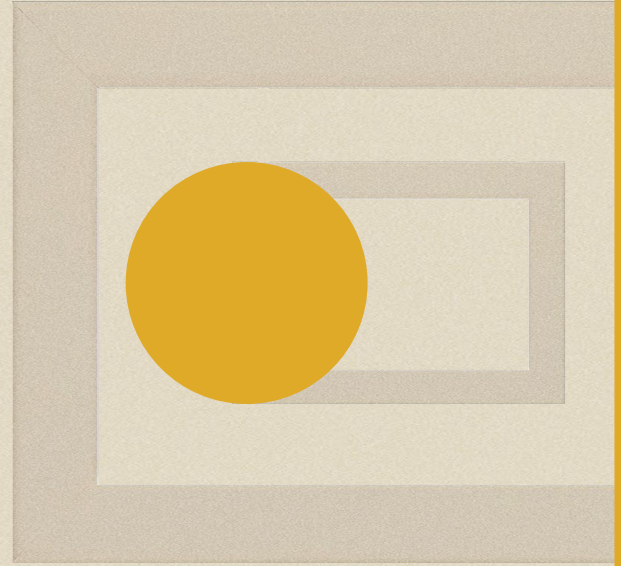
Benchmark

02

METHODOLOG

Y

Writing scripts & running
experiments



Definitions



API Server

a tool that connects the web server to the database



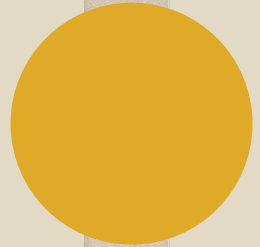
Database

an organized table of mass amounts of data that can be queried



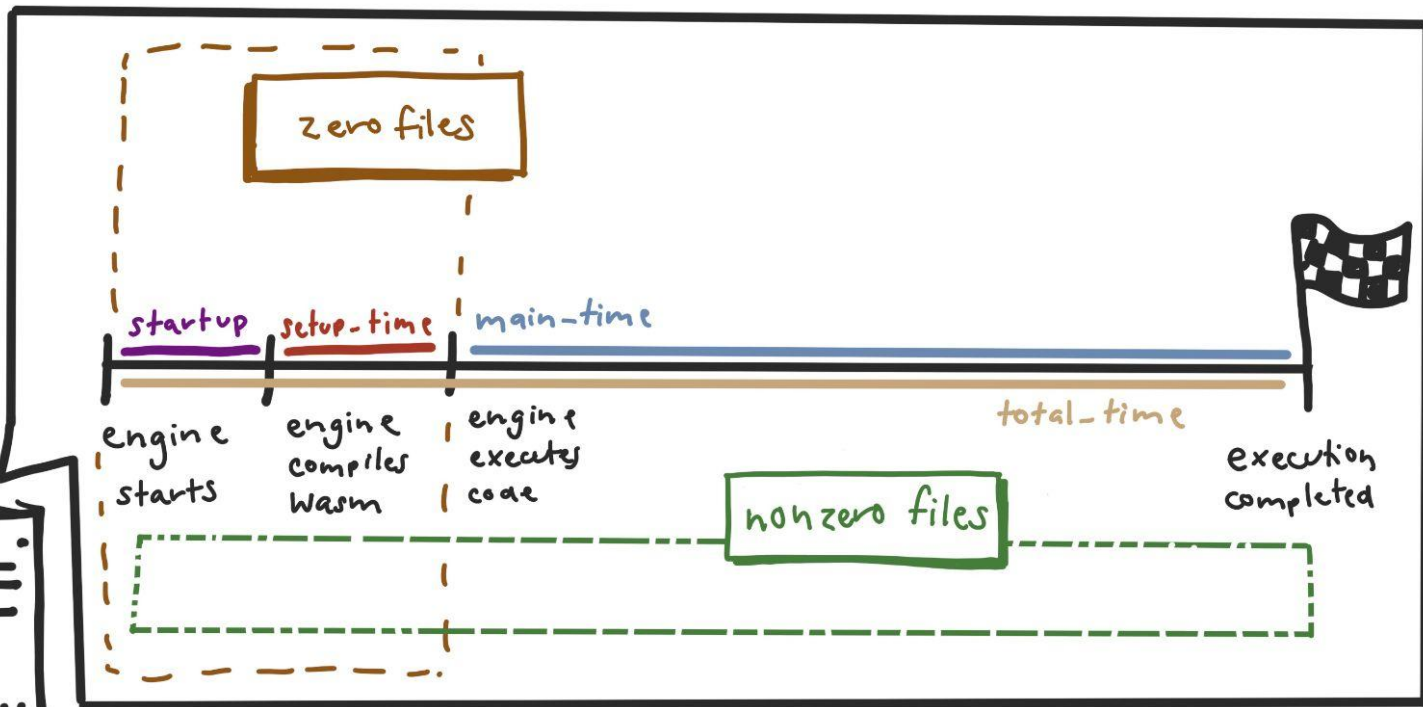
Query

parsing (a database) for data under specified parameters

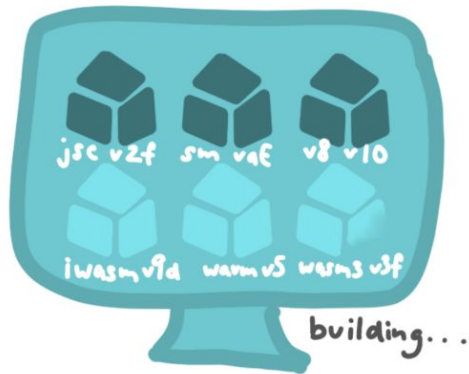


Methodology

- Used `btime` function to time the processes
 - Zero files: times compilation of wasm files with unreachables
 - Nonzero files: times entire operation of wasm files
- Tested 10 engines with different configurations
 - Tested on different versions of the engines
- Examined 3 suites, and 79 wasm files
- Ten runs for each experiment stored as raw data samples
 - Total Time, Main Time, Set Up Time



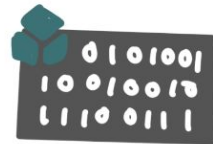
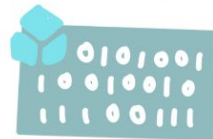
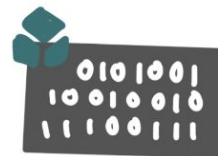
build.py



run.py

ex:

```
def sum():  
    num1=2  
    num2=7  
    return num1 +  
    num2
```



web app
in browser



Request

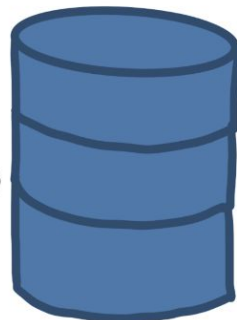


Response

app.py

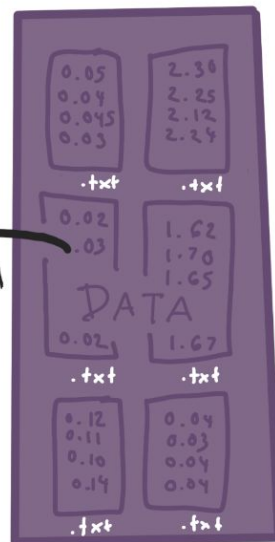


query



DATABASE

upload

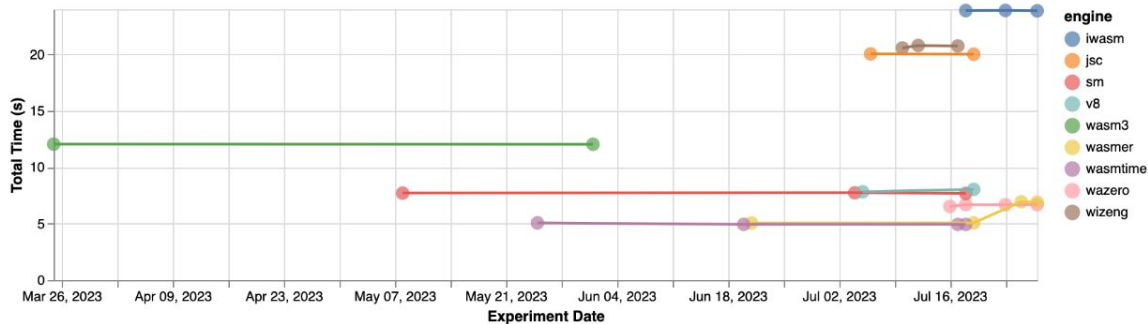


summarize.py

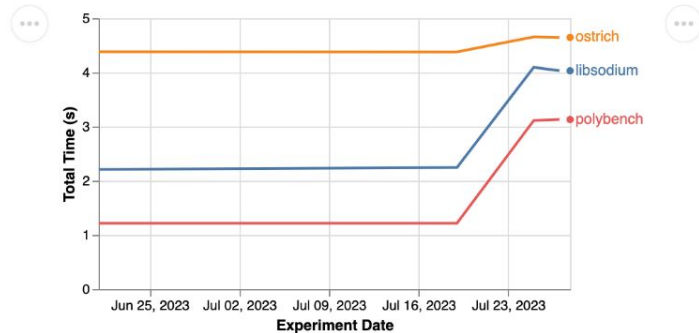
WebAssembly Benchmarking

These graphs demonstrate the time it takes for an engine to startup, compile wasm code, and execute it. Click on the data points to generate a subgraph of the benchmark suites for a specific engine.

Comparison of Engines for Total Time



wasmer Engine



Engines & Configurations

Hover to learn about the engines and their configurations.

JavaScriptCore

JavaScriptCore (jsc) is Apple's web engine for the Safari web browser. We used three tiering configurations for testing:

- **jsc-int:** JITs disabled and rewriting interpreter enabled
- **jsc-bbq:** interpreter enabled and only BBQ quick JIT enabled
- **jsc-omg:** interpreter enabled and only OMG optimizing JIT enabled

V8

V8 is Google's web engine for the Chrome web browser. We used two tiering configurations for testing:

- **v8-liftoff:** only Liftoff baseline JIT enabled
- **v8-turbonfan:** only TurboFan optimizing JIT enabled

SpiderMonkey

SpiderMonkey (sm) is Mozilla's web engine for the Firefox web browser. We used two tiering configurations for testing:

- **sm-base:** only baseline compiler enabled
- **sm-opt:** only optimizing compiler enabled

wasmtime

wasmtime is a non-web WebAssembly engine written in Rust and uses the cranelift optimizing compiler. We tested on wasmtime's default configuration.

Wasm3

Wasm3 is a WebAssembly interpreter and a non-web engine used for embedded systems. We tested with Wasm3's default configuration.

WAVM

WAVM is a non-web engine that uses LLVM to compile WebAssembly code ahead-of-time. We tested with WAVM's default configuration.

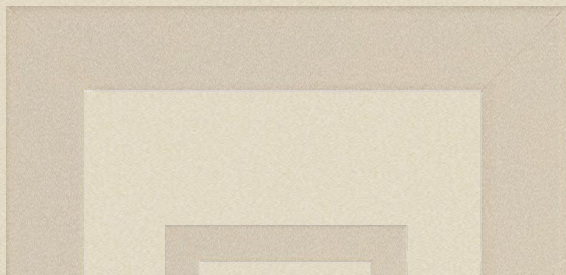
wazero

wazero is a non-web WebAssembly engine written in Go. We tested on wazero's default configuration.

03

RESULTS & EXTENSIONS

Exploring the tradeoff space between Wasm engines

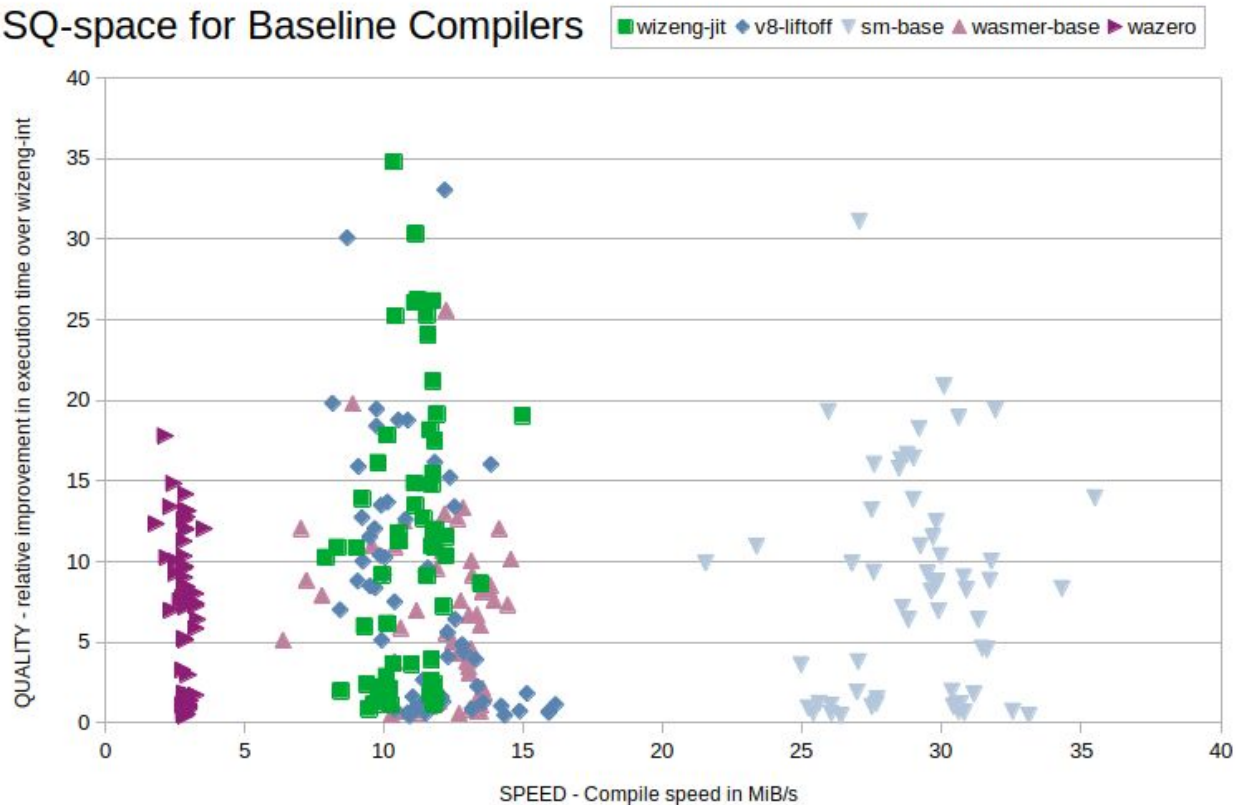


Results & Further Extensions

- Not much data; results currently don't show a lot
 - Ongoing project, current goal is to backfill historical data
- Still tweaking methodology
 - Measurement error: negative time
- Test on different machines
 - 64-bit Linux; now using AWS EC2 Instance Linux
- Create more complex visualizations; more math!
- WebAssembly Research Center
 - Cross collaboration for benchmarking efforts
 - Benchmarks on memory, space, etc.

Expected Results

SQ-space for Baseline Compilers



04

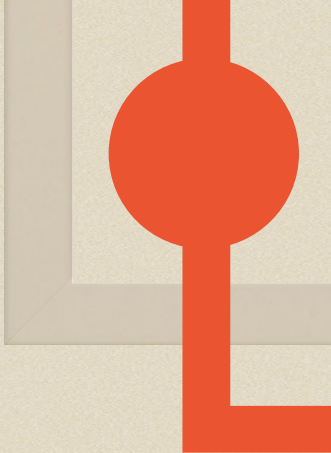
THE RESEARCH EXPERIENCE

CS Research as a Math Major



My Experience

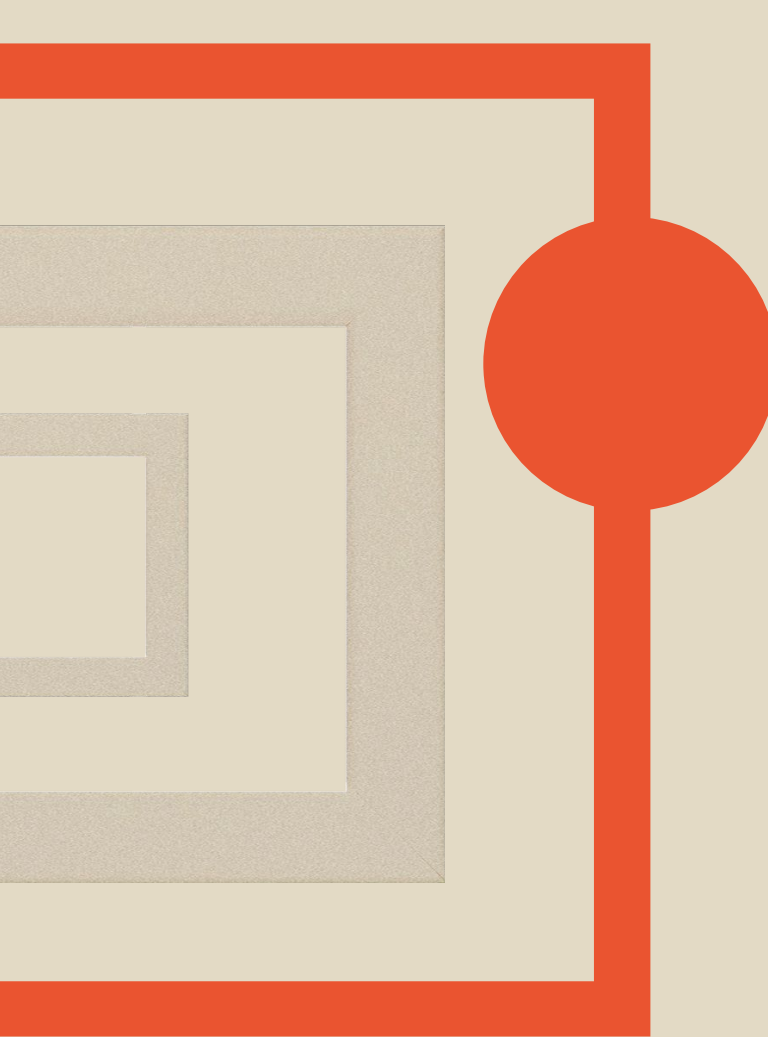
- Learning, learning, and more learning
 - Programming languages, terminology, how to collaborate
 - Improved coding skills
- Collaborative programming
 - Sharing a vision, tackling problems together
- Troubleshooting and problem solving
 - Some questions could not be answered by Google, ChatGPT, nor my mentor
- My priorities: live somewhere new and meet new people



Looking for an REU?

- Consider stipends and meal & travel allowance
- Look to build your professional network
- Cast a wide net, look to simply gain experience
 - Figure out your priorities
- Gain exposure to graduate school and see if it's something you want to pursue
- Meet people who share your passion

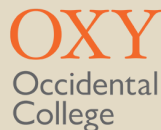




THANK YOU

Contact me at
venenciano@oxy.edu

CREDITS: This presentation template was
created by **Slidesgo**, and includes icons by
Flaticon, and infographics & images by **Freepik**



S3D

Software and Societal
Systems Department